Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester 3)
DSA

Practical 5

Roll No. : **44**	Name: **Amit Umarvaish**
Class : **SYBSC IT**	Batch : **Ist**
Date of Assignment : **22/07/2026**	Date/Time of Submission : **22/07/2026**

Practical 5A

Binary Search Tree: Write a program to:
a. Create a binary search tree.

Code :

```
class TreeNode:
  def __init__(self, data):
  self.data = data
  self.left = None
  self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB
```

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
print("root.right.left.data:", root.right.left.data)
print("Amit 44")

```
class TreeNode:
  def __init__(self, data):
    self.data = data
    self.left = None
    self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
print("root.right.left.data:", root.right.left.data)
print("Amit 44")
```

Output :

```
=== RESTART: C:/Users/IT-47/AppData/Lo
root.right.left.data: E
Amit 44
>
```

Practical 5B

b. Insert nodes into a binary search tree.

Code :

```
class TreeNode:
def __init__(self, data):
  self.data = data
  self.left = None
  self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
print("root.right.left.data:", root.right.left.data)
```

```
def insert(root, data):
    if root is None:
        return TreeNode(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)

    return root

new_node = input("Enter element to insert: ")
root = insert(root, new_node)

print("Node inserted successfully.")
print("Amit 44")
```

Output:

```
root.right.left.data: E
Enter element to insert: I
Node inserted successfully.
Amit 44
```

```
def insert(root, data):
    if root is None:
        return TreeNode(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)

    return root

new_node = input("Enter element to insert: ")
root = insert(root, new_node)

print("Node inserted successfully.")
print("Amit 44")
```

Practical 5C

c. Search for a node in a binary search tree.

Code:

```
class TreeNode:
def __init__(self, data):
  self.data = data
  self.left = None
  self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')
```

```
root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
print("root.right.left.data:", root.right.left.data)

def search(root, key):
    if root is None:
        return False

    print("Checking:", root.data)

    if root.data == key:
        return True

    return search(root.left, key) or search(root.right, key)


key = input("Enter element to search: ")

# Search
if search(root, key):
    print("Element Found")
else:
    print("Element Not Found")
print("Amit 44")
```

```
def search(root, key):
    if root is None:
        return False

    print("Checking:", root.data)

    if root.data == key:
        return True

    return search(root.left, key) or search(root.right, key)


key = input("Enter element to search: ")

# Search
if search(root, key):
    print("Element Found")
else:
    print("Element Not Found")
print("Amit 44")
```

Output:

```
root.right.left.data: E
Enter element to search: G
Checking: R
Checking: A
Checking: C
Checking: D
Checking: B
Checking: E
Checking: F
Checking: G
Element Found
Amit 44
```